

Especificación

Diseño e Implementación de un Lenguaje

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Miguel	Taylor	58286	mtaylor@itba.edu.ar

2.Repositorio

La solución y su documentación serán versionadas en: [Flex-Bison-Compiler](#).

3.Dominio

Desarrollar un lenguaje que permita implementar las funcionalidades básicas de Markdown como así también agregarle nuevas. La idea principal de Markdown es proveer un formato que es fácil de leer y fácil de escribir que luego se convierte en un archivo estructuralmente valido de HTML. Por lo tanto, Markdown es dos cosas: (1) una sintaxis para formatear plain text y (2) una herramienta de software que convierte plain text a HTML. Teniendo esto en cuenta, el lenguaje debe permitir formatear texto proveyendo utilidades comunes de un editor de texto como ejemplo poner títulos, crear listas e insertar imágenes entre otras cosas. Cuando John Gurber creo Markdown estableció como uno de los objetivos principales de diseño que la sintaxis de formateo sea lo más legible posible sin pasar por el compilador. Por lo que un archivo .md debería poder publicarse como está y que el formato visible no impida la lectura de este.

Cuando dicho archivo si es procesado por el programa se espera recibir un documento HTML valido. Lo que resulta en un programa que permite escribir HTML de una manera más fácil. Se debe tener en cuenta que permite escribir una parte del lenguaje completo aceptado por HTML pero este resulta útil ya que actualmente se utilizan en diversos ambientes, por ejemplo para documentación y “README” files, aplicaciones para tomar nota como notion, obsidian y Evernote, entre otras cosas.

4.Construcciones

El lenguaje creado debe ofrecer los siguientes formatos de texto y herramientas:

- (I). Se podrá poner letras dentro del texto en formato de negritas.
- (II). Letras en el estilo de bastardilla.
- (III). Generar títulos y dentro de ellos establecer una jerarquía utilizando variación en el tamaño de cada uno.
- (IV). La creación de “backquotes” siendo estos bloques de texto en los cuales se busca formatearlos de manera tal que le llamen la atención al lector.
- (V). Crear listas con y sin orden. Con la extensión del trabajo se busca también agregar la funcionalidad de listas ordenadas utilizando letras como índice.
- (VI). Insertar código y que este se renderice de manera tal. Esta función debe ofrecer inserción de código inline (un recorte corto de código) dentro de un párrafo como también insertar un bloque multi línea de código.
- (VII). La inserción de líneas horizontales como separador de contenido.
- (VIII). Añadir hipervínculos al texto
- (IX). Insertar imágenes y el texto alternativo que se utiliza para dar una descripción de la imagen para personas con visión imitada.
- (X). Deberá poder crear tablas.
- (XI). Crear una bibliografía del documento utilizando todas las instancias en las cuales se les ingreso un superíndice con información al texto en el orden que fueron insertadas.
- (XII). Implementar listas del tipo “checklists”.
- (XIII). Insertar emojis utilizando emoji shortcodes.
- (XIV). La posibilidad de resaltar texto.
- (XV). Manejo de sub índices.

Siendo las nueve primeras funcionalidades básicas de Markdown y las últimas siete siendo una extensión al lenguaje que se propone agregar mediante el trabajo.

5.Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que utilice negritas y letras en bastardilla.
- (II). Probar todas las jerarquías de titulo.
- (III). Crear un “backquote” entre dos párrafos

- (IV). Probar las listas con y sin orden en combinación a que algunos artículos de la lista estén en negritas.
- (V). Crear una tabla de cinco columnas y dos filas en las cuales algunas celdas estén vacías.
- (VI). Insertar línea horizontal como división de un párrafo y el siguiente título.
- (VII). Crear una bibliografía con tres elementos.
- (VIII). Insertar una imagen con su respectivo texto alternativo.
- (IX). Insertar un bloque de código de Python.
- (X). Crear un checklist con dos elementos ya completos y uno sin completar.

En cuanto a los casos de rechazo, estos no impedirán que el documento .md sea procesado y resulte en un HTML válido sino que lo que sucederá es que el formato deseado no se aplicará ya que lo escrito aparecerá dentro de un párrafo donde se ve la sintaxis del lenguaje incluyendo el error que evita el parseo correcto. Los siguientes son casos de **rechazo**:

- (I). Un programa donde busca insertar un hipervínculo y le falta cerrar un paréntesis.
- (II). Un subíndice que no cierra.
- (III). El uso de un emoji shortcode inexistente.
- (IV). No cerrar el área de texto resaltado.
- (V). Crear un bloque de código sobre un lenguaje inexistente.

6. Ejemplos

Apuntes de materia de base de datos en los cuales se puede ver la jerarquía de títulos y el uso de letras en negrita como en bastardilla:

```
#Base de Datos
```

```
###Algebra relacional
```

```
Se basa en la teoría de conjuntista matemática. Al ser basado en los conceptos matemáticos, esto permite la _anidación de relaciones._
```

```
#### Operadores
```

1. **selección:**
2. **proyección:**

Crear una tabla y debajo también se genera una lista del tipo “checklist”:

```
| X | F(x)|
```

```
|--|---
```

```
|1|2|
```

```
|2|3|
```

Guías a completar:

- ☒ TP1 Introducción
- ☒ TP2 Gramáticas
- ☒ TP3 Lenguajes Regulares Autómatas
- ☐ TP4 Lenguajes Regulares Expresiones Regulares